Goals of

Distributed Systems

Resource Sharing • Transparency • Openness • Scalability • Reliability • Performance

Lecture Outline

What we will cover today

1. Core Goals of Distributed Systems

- Resource Sharing
- Openness
- Scalability
- Reliability & Performance

2. Transparency in Depth

- Definition of Transparency
- Access & Location
- Replication, Migration
- Failure transparency

Learning Objective

Students will understand the fundamental goals that drive the design of distributed systems.

Students will be able to explain the different types of transparency and their trade-offs.

Students will learn how to evaluate distributed systems based on reliability and performance metrics.

The Main Goals of Distributed Systems

Overview of the 6 key goals

Goal	Description	Key Benefit
1. Resource Sharing	Access and use remote resources across the network	Cost-effective collaboration
2. Transparency	Hide the detail that resources are distributed	Simplifies usage for users/apps
3. Openness	System can be extended with well-defined interfaces	Interoperability & flexibility
4. Scalability	Remains effective as users and resources increase	Handles growth gracefully
5. Reliability	System continues to function despite failures	High availability
6. Performance	Quick to respond, high throughput, meets QoS deadlines	Good user experience

Making Resources Accessible

Goal 1

Resource Sharing: The ability to access hardware, software, and data across a network.

Types of Resources:

- Hardware: Printers, computers, storage.
- Software/Data: Files, web pages, databases.
- Networks: Internet, Wi-Fi, communication channels.

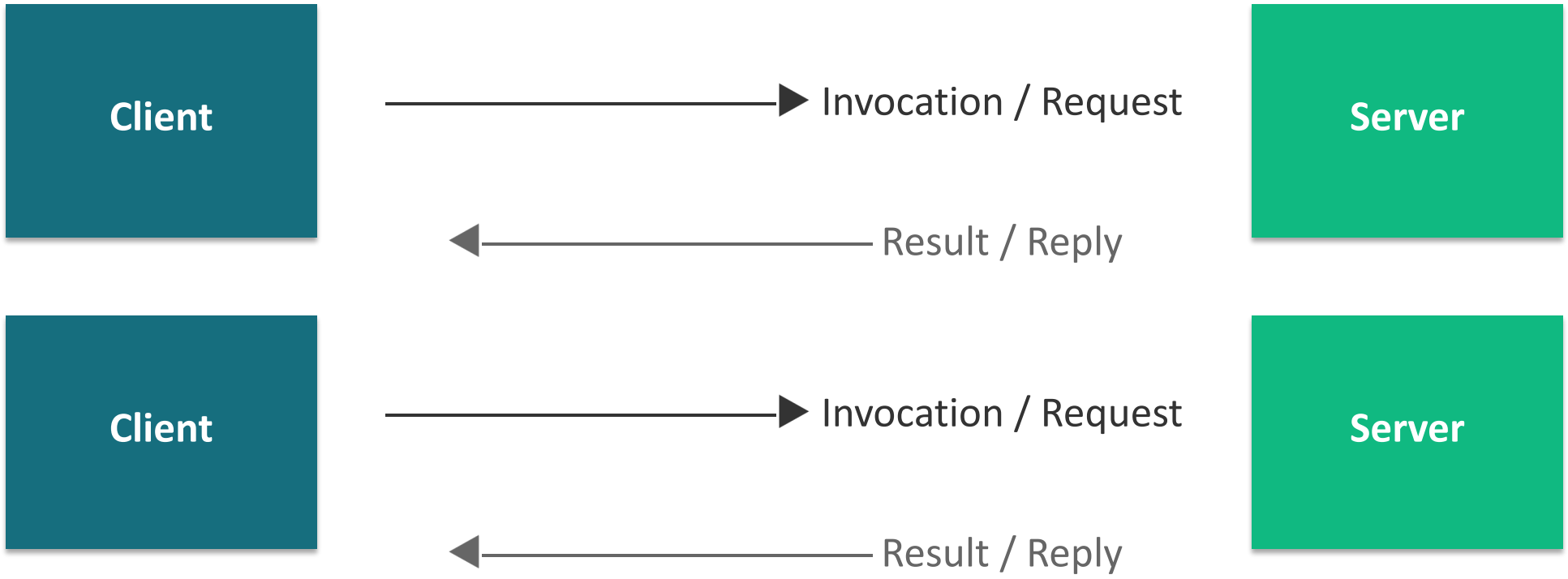
Reasons for Sharing:

- Economics: Save money (share 1 printer instead of 20).
- Collaboration: People can work together even when they are in different places. (Google Docs, Zoom).
- Commerce: Support online services (Amazon, Banking).

Mechanism: Client-Server Computing

Clients request services from servers. For example, a web browser (client) requests web pages from a web server. Servers manage resources and respond to client requests.

Resource Sharing: Client-Server Model



Key Points:

- Servers are passive (wait for requests), clients are active (make requests).

Goal 2: Transparency

Definition & Importance

***Transparency** is the ability of a distributed system to hide its distributed nature so that users understanding it as a single unified system.*

Transparency Hides:

- Where processes are located.
- Where resources are stored.
- How components communicate.
- Whether resources are replicated, or if components fail and recover.
- Whether multiple users access resources simultaneously.

Why is Transparency Important?

- Allows users to use the system without worrying about where resources are located.
- Makes programming easier by hiding network details.
- Enables system administrators to change hardware/software without affecting users.
- Provides a 'single system image'.

Different Types of Transparency

Transparency Type	What It Hides	Example
Access	Hide differences in data formats and access methods. Users access local and remote resources identically.	Same file operations for local & remote files
Location	Hide where a resource is physically located. Users use logical names (URLs) not IP addresses.	URL doesn't reveal server's IP/location
Replication	Hide that multiple copies of a resource exist. Users see only one copy (e.g., Netflix movies on many servers).	User sees one database, not its backups
Migration	Hide that a resource moves to another location. The name or address stays valid.	Server moves datacenters, URL stays same
Relocation	Hide that a resource moves while it is currently being used (e.g., switching Wi-Fi towers during a call).	Laptop moves between WiFi cells, call stays active

Different Types of Transparency

Transparency Type	What It Hides	Example
Failure	Hide the failure and recovery of a resource. Users continue working normally.	Server crashes, backup takes over without a problem
Concurrency	Hide that multiple users use the same resource simultaneously. Data must remain consistent.	Multiple users edit a doc without interference

Access Transparency

Enabling local and remote resources to be accessed identically

Access Transparency : Enables local and remote resources to be accessed using identical operations.

What it hides:

- Differences in machine architectures (e.g., Intel x86 vs. ARM).
- Differences in operating systems (e.g., Windows vs. Linux file paths).
- Differences in data representation (e.g., Big-endian vs. Little-endian, ASCII vs. Unicode).

How it is achieved:

- Middleware provides standard data representations (e.g., CORBA, JSON, XML).
- proxies translate local calls into request messages (A proxy acts as a helper that makes a remote service look like a local one).

Example: provide the same required resources whether the files for example inside the folder are stored on the local hard drive or on a remote network server.

Location Transparency

Accessing resources without knowledge of their location

Location Transparency : Enables resources to be accessed without knowledge of their physical location.

What it hides:

- The physical location (building, city, or IP address) of the server hosting the resource.
- The network route taken to reach the resource.

How it is achieved:

- Using logical names (e.g., URLs, domain names) instead of IP physical network addresses.
- Name services (like DNS) translate logical names to network IP addresses at runtime.

Example: www.google.com hides the actual server location .

Replication Transparency

Using multiple instances of resources without knowing

Replication Transparency: Enables multiple copies of resources to be used to increase reliability and performance without users or application programmers being aware of the duplicate copies.

Why we need Replication:

- Reliability: If one server fails, another replica can serve the request.
- Performance: Place a copy of the resource closer to the user (e.g., Content Delivery Network CDN).
- Load Balancing: Distribute requests among multiple replicas.

Note: A **replica** is a copy of a resource that contains the same data or functionality as another resource.

What it hides:

- The existence of multiple copies of a data object.
- The detail that a request might be routed to replica A today and replica B tomorrow.

***Example:** A user interacts with Google as if it were a single system, while the underlying infrastructure consists of many replicated servers that remain hidden from the user.*

Migration Transparency

Allowing movement of resources and clients

Migration Transparency: Allows the movement of resources and clients within a system without affecting the operation of users or programs.

What it hides:

- That a resource has been moved from one physical location to another.
- That a client has moved to a new network or environment (e.g., mobile roaming).

How it is achieved:

- Using location-independent identifiers (e.g., A file keeps the same unique ID even after it is moved to a different server, so users can still find it without knowing its new location.).
- Employing forwarding pointers (When a mobile device moves to a new network, a forwarding service automatically redirects messages to the device's new location.).

Example: A person is talking on a mobile phone while traveling by train from **London to Paris**. The phone automatically switches between different cell towers and networks along the journey, so the call does not disconnect.

Failure Transparency

Concealing faults and allowing tasks to complete

Failure Transparency: Enables the hiding of faults, allowing users and application programs to complete tasks despite the failure of hardware or software components.

What it hides:

- That a server has crashed or a network link has failed.
- The recovery process (e.g., retransmitting lost messages).
- The switching over to a backup replica.

The Main Challenge

- **It is very difficult to distinguish between a dead resource and a still alive and functioning, but taking a very long time to respond**
- **If a server doesn't reply, is it crashed, or just overloaded?**

***Example:** A database transaction either fully completes or rolls back entirely(if the server crashes during processing time).*

Goal 3: Openness

Extensibility and published interfaces

Openness means that different systems and application can work together easily because they follow common standards and rules.

Simple Example

- A user can send an email from Gmail to Outlook because both email systems use common standards. This is an example of openness.
- A Windows computer can access a website hosted on a Linux server because both use standard Internet protocols. This demonstrates openness.

Benefits of Openness

- Interoperability: Components from different vendors work together.
- Portability: Applications can run on different systems without modification((run on multiple platforms)).
- Extensibility (easy to add/replace components).

Openness: The Role of Middleware

Architecture

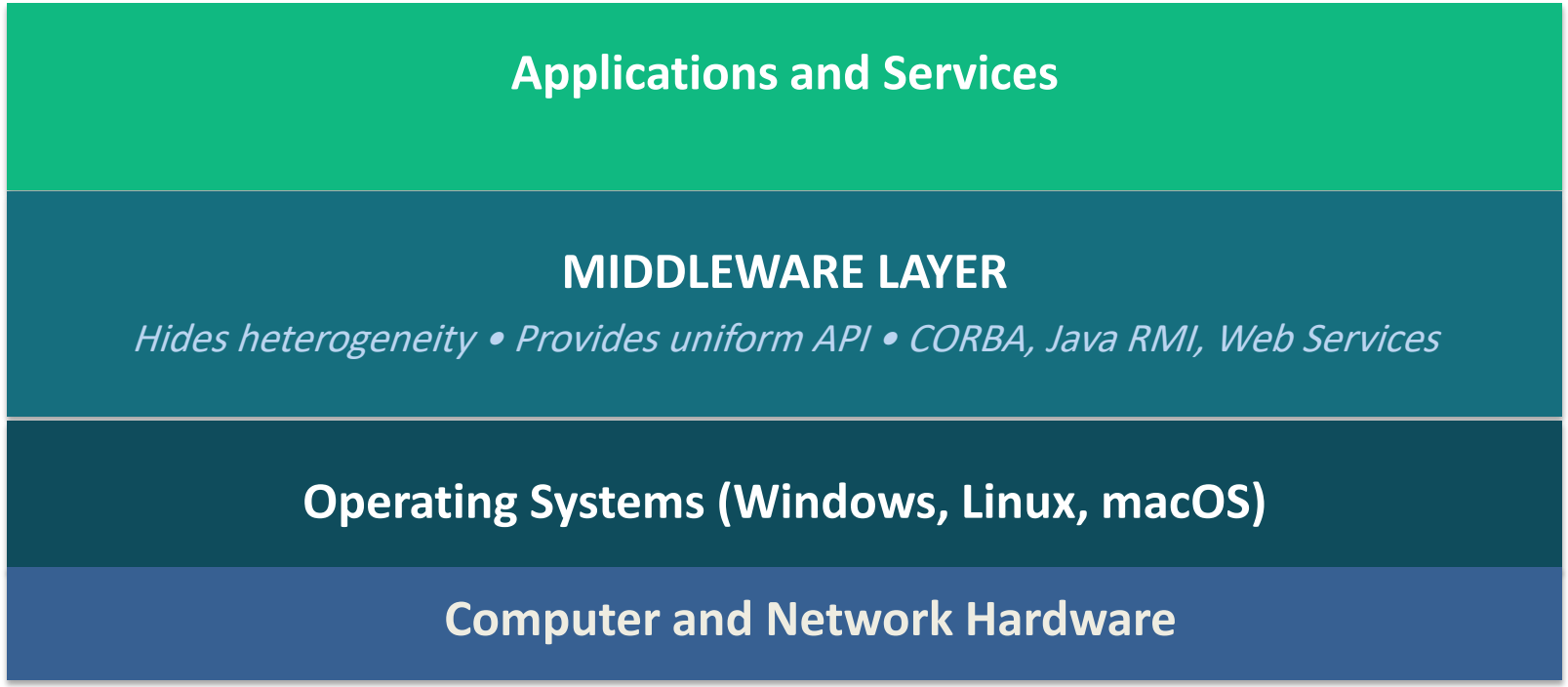
Middleware acts as a bridge between applications and different OS/hardware.

Functions:

- Hides Heterogeneity: Windows, Linux, macOS work together.
- Provides Uniform API: Same programming interface everywhere.

  Examples: CORBA, Java RMI, Web Services (REST/SOAP).

Allows a Java client to invoke a method on a C++ server without any problem.



Scalability

Goal 4

Scalability: A system is scalable if it remains effective when the number of users and resources increases significantly.

Three Dimensions of Scalability:

1. Size Scalability: Add more users and resources easily (e.g., Netflix serving millions).
2. Geographical Scalability: Users and resources may lie far apart (e.g., global servers).
3. Administrative Scalability: Still easy to manage across multiple organizations (e.g., universities sharing resources).

Problem: A system scalable in one dimension often or may be has a loss of performance as it scales up.

Scaling Techniques (1/2)

Goal 4 · Scalability

Technique 1: Hide Communication Latencies

- Use asynchronous communication (send and continue working).
- Move computation to the client (e.g., JavaScript form validation instead of server-side).

Technique 2: Distribution (Partitioning)

- Split a large component into smaller parts and spread them across the system.

Example: DNS name space divided into zones, each handled by a single name server.

Example: Web is physically distributed across thousands of servers.

Benefit: Divides the workload. No single server stores everything.

Technique 3: Replication and Caching

Replication: Make copies of data/services closer to users(Increases availability and balances load).

Caching: A special form of replication(Temporary copy made by client on demand).

Benefits: Faster response, lower latency, reduced network traffic.

Goal 5: Reliability

Handling failures gracefully

Reliability : The ability of a system to continue operating correctly despite failures.

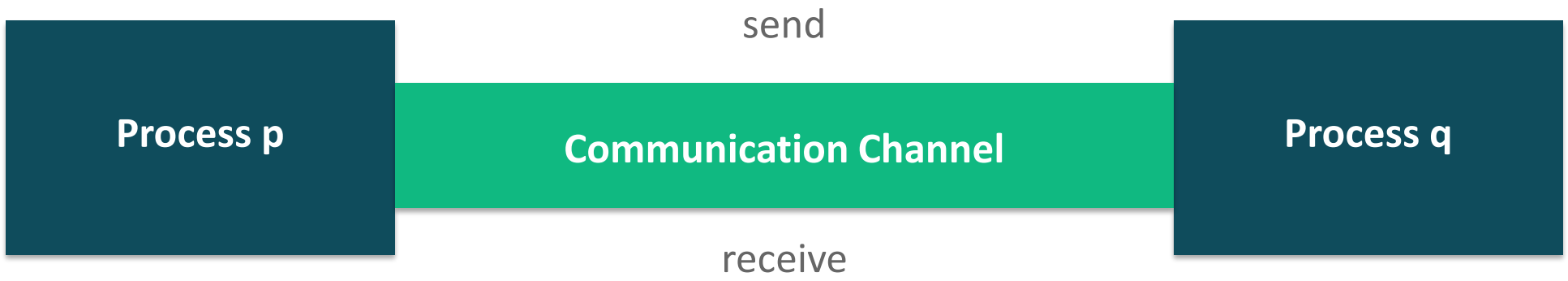
Techniques for Dealing with Failures:

Technique	Description
Detecting Failures	Using checksums to detect corrupted data; using timeouts to detect crashes.
Masking Failures	Retransmitting lost messages; storing data in multiple disks (RAID) to mask disk failure.
Tolerating Failures	inform users and retry later if a server is unresponsive.
Recovery	Restoring consistent state using logs and rollback (e.g., in databases).
Redundancy	Using multiple routers, replicated DNS servers, or backup power supplies.

RAID (Redundant Array of Independent Disks) is a technique that uses multiple disks

Reliability: Types of Failures

Classifying failures (Coulouris et al., p.67)



Failure Type	What Happens
Omission (Process)	Process crashes (halts and stays halted).
Omission (Channel)	Message is dropped due to buffer overflow or network error.
Arbitrary (Byzantine)	Process behaves randomly or incorrectly, giving wrong information rather than simply stopping.
Timing	Response takes longer than the stated time bound (in synchronous systems).

Why Important? Each failure needs a different solution (backup, retransmission, voting, timeouts).

Example: Bank Server Crash (Process), Transfer Request Lost (Channel), Hacked Server (Byzantine).

Goal 6: Performance

Responsiveness and throughput

Performance : The ability of a system to complete tasks quickly, meet timing deadlines, and provide high throughput.

Key Performance Metrics:

Metric	Description	Why it matters
Latency	Delay between initiating an operation and the result arriving.	Affects user feeling (e.g., web page load time).
Throughput	The rate at which data is processed or transmitted (e.g., Mbps).	Determines how much work the system can handle.
Jitter	Variation in the time taken to deliver a series of messages.	Essential for multimedia (audio/video distortion).
Response Time	Total time to respond to a user request.	Affect feeling of user the system fast or slow

Case Study: The World Wide Web

Applying the 6 Goals

The Web is the ultimate large-scale distributed system. How does it meet our 6 goals?

Goal	How the Web Achieves It
Resource Sharing	Clients (browsers) request web pages from servers via HTTP. Shares information globally.
Transparency	Location Transparency: URLs hide IP addresses. Access Transparency: Same browser UI for all servers.
Openness	Based on published standards (HTTP, HTML, URL). Browsers and servers can be from any vendor.
Scalability	Uses DNS hierarchy for naming. Web Proxy Caching and CDNs (like Akamai) handle massive load.
Reliability	TCP protocol retransmits lost packets. Web farms replicate servers for high availability.
Performance	HTTP 2.0 and caching reduce latency. Browser opens multiple connections to fetch images in parallel.

Case Study: Transparency in the Web

Examining Web URLs

URL: http://www.example.com/index.html

Breakdown of the URL:

- http:// → Scheme (Protocol used for access)
- www.example.com → Server name (Domain name, hiding location)
- /index.html → Path name (Specific resource on the server)

Transparency Achieved

- Location Transparency: The domain name hides the actual IP address and physical server location.
- Access Transparency: The browser handles HTTP regardless of the underlying OS or network.

Trade-offs Between Goals

Balancing competing requirements

It is impossible to perfectly achieve all goals simultaneously. Designers must make trade-offs.

Trade-off	Description
Transparency vs. Performance	When a system tries to hide the delay caused by communicating with remote servers, it often needs extra work and resources. This can make the system slower and more costly.
Reliability vs. Performance	Reliability often requires acknowledging every message and keeping replicas in sync, which hurts performance.
Scalability vs. Security	When a system is spread across many computers and can grow easily, it becomes more difficult to control security
Openness vs. Performance	Using open and standard formats helps different systems work together, but it can make processing slower because extra work is needed to read and interpret the data.

Trade-offs Between Goals

Balancing competing requirements

- Every distributed system design must arrange goals based on its specific application domain (e.g., banking selects reliability; gaming selects performance).

Summary

1. The 6 main goals of distributed systems are Resource Sharing, Transparency, Openness, Scalability, Reliability, and Performance.
2. Transparency hides the distributed nature of the system. The 5 key types are Access, Location, Replication, Migration, and Failure.
3. Openness relies on published interfaces and middleware to allow heterogeneous systems to work together.
4. Scalability requires avoiding centralized bottlenecks by using techniques like DNS distribution, caching, and replication.
5. Reliability is achieved through failure detection, masking, tolerance, and redundancy.
6. Performance is measured by latency, throughput, and jitter. QoS guarantees are needed for real-time multimedia.
7. Designers must constantly balance trade-offs between these goals (e.g., Transparency vs. Performance).